Python Pandas

Python Pandas is defined as an open-source library that provides high-performance data manipulation in Python. This tutorial is designed for both beginners and professionals.

Data analysis requires lots of processing, such as **restructuring, cleaning** or **merging**, etc. There are different tools are available for fast data processing, such as **Numpy, Scipy, Cython**, and **Panda**. But we prefer Pandas because working with Pandas is fast, simple and more expressive than other tools.

Pandas is built on top of the **Numpy** package, means **Numpy** is required for operating the Pandas.

Python Pandas Data Structure

The Pandas provides two data structures for processing the data, i.e., **Series** and **DataFrame**, which are discussed below:

1) Series

It is defined as a one-dimensional array that is capable of storing various data types. The row labels of series are called the **index**. We can easily convert the list, tuple, and dictionary into series using "series' method. A Series cannot contain multiple columns. It has one parameter:

Data: It can be any list, dictionary, or scalar value.

Creating Series from Array:

Before creating a Series, Firstly, we have to import the numpy module and then use array() function in the program.

```
import pandas as pd
import numpy as np
info = np.array(['P','a','n','d','a','s'])
a = pd.Series(info)
print(a)
```

Output

```
0   P
1   a
2   n
3   d
4   a
5   s
dtype: object
```

Create a Series from dict

We can also create a Series from dict. **If the dictionary object is being passed as an input and the index is not specified, then the dictionary keys are taken in a sorted order to construct the index**.

If index is passed, then values correspond to a particular label in the index will be extracted from the **dictionary**.

Example:

```
#import the pandas library

import pandas as pd

import numpy as np

info = {'x' : 0., 'y' : 1., 'z' : 2.}

a = pd.Series(info)

print (a)
```

Output:

```
x    0.0
y    1.0
z    2.0
dtype: float64
```

Create a Series using Scalar:

If we take the scalar values, then the index must be provided. The scalar value will be repeated for matching the length of the index.

Example:

```
#import pandas library

import pandas as pd

import numpy as np

x = pd.Series(4, index=[0, 1, 2, 3])

print (x)
```

Output

```
0    4
1    4
2    4
3    4
dtype: int64
```

Accessing data from series with Position:

Once you create the Series type object, you can access its indexes, data, and even individual elements.

The data in the Series can be accessed similar to that in the ndarray.

Example:

```
import pandas as pd

x = pd.Series([1,2,3],index = ['a','b','c'])

#retrieve the first element

print (x[0])
```

Output:

1

Example:

```
import numpy as np

import pandas as pd

fruits=['apples','oranges','cherries','pears']

s1=pd.Series([20,30,52,10],index=fruits)

print(s1)

s2=pd.Series([17,13,31,32],index=fruits)

print(s2)

print(s1+s2)

print("sum of s1:",sum(s1))

print("sum of s2:",sum(s2))

print("sum of s1 and s2:",sum(s1+s2))
```

output:

```
apples      20
oranges     30
cherries    52
pears       10
dtype: int64
apples      17
oranges     13
cherries    31
pears       32
dtype: int64
apples      37
oranges     43
```

```
cherries    83
pears       42
dtype: int64
sum of s1: 112
sum of s2: 93
sum of s1 and s2: 205
```

Example:

```
import numpy as np

import pandas as pd

fruits=['apples','oranges','cherries','pears']

s1=pd.Series([20,30,52,10],index=fruits)

print(s1)

s2=pd.Series([17,13,31,32],index=fruits)

print(s2)

print(s1+s2)

print("min of s1:",min(s1))

print("min of s2:",min(s2))

print("min of s1 and s2:",min(s1+s2))


print("max of s1:",max(s1))

print("max of s2:",max(s2))

print("max of s1 and s2:",max(s1+s2))


print("count of s1:",len(s1))
```

```
print("count of s2:",len(s2))

print("count of s1 and s2:",len(s1+s2))
```

output:

```
apples      20
oranges     30
cherries    52
pears       10
dtype: int64
apples      17
oranges     13
cherries    31
pears       32
dtype: int64
apples      37
oranges     43
cherries    83
pears       42
dtype: int64
min of s1: 10
min of s2: 13
min of s1 and s2: 37
max of s1: 52
max of s2: 32
max of s1 and s2: 83
count of s1: 4
count of s2: 4
count of s1 and s2: 4
```

Example:

```
import numpy as np

import pandas as pd

fruits=['peaches','oranges','cherries','pears']

fruits2=['rasp berries','oranges','cherries','pears']
```

```
s1=pd.Series([20,33,52,10],index=fruits)

s2=pd.Series([17,13,31,62],index=fruits2)

print(s1+s2)
```

output:

```
cherries       83.0
oranges        46.0
peaches         NaN
pears          72.0
rasp berries     NaN
dtype: float64
```

Pandas Accessing multiple items:

```
import numpy as np

import pandas as pd

fruits=['peaches','oranges','cherries','pears']

s = pd.Series([20,33,52,10],index=fruits)

print(s[['oranges']])

print(s[['oranges','cherries','pears']])

print((s+3) *4)

print(s)

print("===========")

print(np.sin(s))
```

Output:

```
oranges    33
dtype: int64
oranges     33
cherries    52
pears       10
dtype: int64
peaches      92
oranges     144
```

```
cherries    220
pears        52
dtype: int64
peaches     20
oranges     33
cherries    52
pears       10
dtype: int64
===========
peaches     0.912945
oranges     0.999912
cherries    0.986628
pears      -0.544021
dtype: float64
```

Example:

```
import numpy as np

import pandas as pd

cities = {"London" : 8615246,

          "Berlin" : 3562166,

          "Madrid" : 3165235,

          "Rome"   : 2874038,

          "Paris"  : 2273305,}

city_Series = pd.Series(cities)

print(city_Series)

print(city_Series["Madrid"])
```

Output:

```
London    8615246
Berlin    3562166
Madrid    3165235
Rome      2874038
Paris     2273305
```

```
dtype: int64
3165235
```

Pandas Series.to_frame()

Series is defined as a type of list that can hold an integer, string, double values, etc. It returns an object in the form of a list that has an index starting from 0 to n where n represents the length of values in Series.

The main difference between Series and Data Frame is that **Series can only contain a single list with a particular index, whereas the DataFrame is a combination of more than one series that can analyze the data**.

The Pandas **Series.to_frame()** function is used to convert the series object to the DataFrame.

Example:

```
import pandas as pd

import numpy as np

s = pd.Series(["a","b","c"],name = 'Vals')

s.to_frame()

print(s)
```

Output:

```
0    a
1    b
2    c
Name: Vals, dtype: object
```

Python Pandas DataFrame

It is a widely used data structure of pandas and works with a two-dimensional array with labeled axes (rows and columns). DataFrame is defined as a standard way to store data and has two different indexes, i.e., row index and column index.

Create a DataFrame using List:

We can easily create a DataFrame in Pandas using list.

```
import pandas as pd
# a list of strings
x = ['Python', 'Pandas']

# Calling DataFrame constructor on list
df = pd.DataFrame(x)
print(df)
```

Output

```
     0
0   Python
1   Pandas
```

Example

```
import pandas as pd

import numpy as np

#Create a DataFrame

info = {

    'Name':['Parker','Smith','John','William'],

    'sub1_Marks':[52,38,42,37],

    'sub2_Marks':[41,35,29,36]

    }

data = pd.DataFrame(info)

print(data)
```

```
# standard deviation of the dataframe
print(data.std())
print(data.mean())
```

output:

```
     Name  sub1_Marks  sub2_Marks
0   Parker         52         41
1    Smith         38         35
2     John         42         29
3  William         37         36
sub1_Marks    6.849574
sub2_Marks    4.924429
dtype: float64
sub1_Marks    42.25
sub2_Marks    35.25
dtype: float64
```

Create a DataFrame from Dict of Series:

```
import pandas as pd
import numpy as np
info = {'one' : pd.Series([1, 2, 3, 4, 5, 6], index=['a', 'b', 'c', 'd', 'e', 'f']),
            'two' : pd.Series([1, 2, 3, 4, 5, 6, 7, 8], index=['a', 'b', 'c', 'd', 'e', 'f', 'g',
'h'])}

df = pd.DataFrame(info)
print (df)
```

Output:

```
   one  two
a  1.0    1
b  2.0    2
c  3.0    3
d  4.0    4
e  5.0    5
f  6.0    6
g  NaN    7
h  NaN    8
```

Column Selection

```
import pandas as pd

import numpy as np

info = {'one' : pd.Series([1, 2, 3, 4, 5, 6], index=['a', 'b', 'c', 'd', 'e', 'f']),

            'two' : pd.Series([1, 2, 3, 4, 5, 6, 7, 8], index=['a', 'b', 'c', 'd', 'e', 'f', 'g',
'h'])}


df = pd.DataFrame(info)

print (df['one'])
```

Output:

```
a    1.0
b    2.0
c    3.0
d    4.0
e    5.0
f    6.0
g    NaN
h    NaN
Name: one, dtype: float64
```

Column Addition

We can also add any new column to an existing DataFrame.

Example:

```
import pandas as pd

import numpy as np

info = {'one' : pd.Series([1, 2, 3, 4, 5, 6], index=['a', 'b', 'c', 'd', 'e', 'f']),

            'two' : pd.Series([1, 2, 3, 4, 5, 6, 7, 8], index=['a', 'b', 'c', 'd', 'e', 'f', 'g',
'h'])}


df = pd.DataFrame(info)

print ("Add new column by passing series")

df['three']=pd.Series([20,40,60],index=['a','b','c'])

print (df)

print ("Add new column using existing DataFrame columns")

df['four']=df['one']+df['three']

print (df)
```

Output:

```
Add new column by passing series
   one  two  three
a  1.0    1   20.0
b  2.0    2   40.0
c  3.0    3   60.0
d  4.0    4    NaN
e  5.0    5    NaN
f  6.0    6    NaN
g  NaN    7    NaN
h  NaN    8    NaN
Add new column using existing DataFrame columns
   one  two  three  four
a  1.0    1   20.0  21.0
b  2.0    2   40.0  42.0
c  3.0    3   60.0  63.0
d  4.0    4    NaN   NaN
e  5.0    5    NaN   NaN
f  6.0    6    NaN   NaN
```

```
g  NaN    7    NaN   NaN
h  NaN    8    NaN   NaN
```

Column Deletion:

We can also delete any column from the existing DataFrame.

Example:

```
import pandas as pd

import numpy as np

info = {'one' : pd.Series([1, 2, 3, 4, 5, 6], index=['a', 'b', 'c', 'd', 'e', 'f']),

            'two' : pd.Series([1, 2, 3, 4, 5, 6, 7, 8], index=['a', 'b', 'c', 'd', 'e', 'f', 'g',
'h'])}


df = pd.DataFrame(info)

print ("Add new column by passing series")

df['three']=pd.Series([20,40,60],index=['a','b','c'])

print (df)

print ("Add new column using existing DataFrame columns")

df['four']=df['one']+df['three']

print (df)

del df['one']

print(df)
```

Output:

```
Add new column by passing series
   one  two  three
a  1.0    1   20.0
b  2.0    2   40.0
c  3.0    3   60.0
d  4.0    4    NaN
e  5.0    5    NaN
```

```
f  6.0    6    NaN
g  NaN    7    NaN
h  NaN    8    NaN
Add new column using existing DataFrame columns
   one  two  three  four
a  1.0    1   20.0  21.0
b  2.0    2   40.0  42.0
c  3.0    3   60.0  63.0
d  4.0    4    NaN   NaN
e  5.0    5    NaN   NaN
f  6.0    6    NaN   NaN
g  NaN    7    NaN   NaN
h  NaN    8    NaN   NaN
   two  three  four
a    1   20.0  21.0
b    2   40.0  42.0
c    3   60.0  63.0
d    4    NaN   NaN
e    5    NaN   NaN
f    6    NaN   NaN
g    7    NaN   NaN
h    8    NaN   NaN
```

Example 2:

```
import pandas as pd

 info = {'one' : pd.Series([1, 2], index= ['a', 'b']),

            'two' : pd.Series([1, 2, 3], index=['a', 'b', 'c'])}

df = pd.DataFrame(info)

print ("The DataFrame:")

print (df)

# using del function

print ("Delete the first column:")

del df['one']

print (df)
```

```
# using pop function

print ("Delete the another column:")

df.pop('two')

print (df)
```

Output:

```
The DataFrame:
   one  two
a  1.0    1
b  2.0    2
c  NaN    3
Delete the first column:
   two
a    1
b    2
c    3
Delete the another column:
Empty DataFrame
Columns: []
Index: [a, b, c]
```

Row Selection, Addition, and Deletion

Row Selection:

We can easily select, add, or delete any row at anytime. First of all, we will understand the row selection.

Selection by Label:

We can select any row by passing the row label to a **loc** function.

Example:

```
import pandas as pd

info = {'one' : pd.Series([1, 2, 3, 4, 5], index=['a', 'b', 'c', 'd', 'e']),

            'two' : pd.Series([1, 2, 3, 4, 5, 6], index=['a', 'b', 'c', 'd', 'e', 'f'])}

df = pd.DataFrame(info)
```

```
print (df.loc['b'])
```

Output:

```
one    2.0
two    2.0
Name: b, dtype: float64
```

Example 2

```
 import pandas as pd
info = {'one' : pd.Series([1, 2, 3, 4, 5], index=['a', 'b', 'c', 'd', 'e']),
            'two' : pd.Series([1, 2, 3, 4, 5, 6], index=['a', 'b', 'c', 'd', 'e', 'f'])}
df = pd.DataFrame(info)
print(df)
print (df.loc['b'])
```

Output:

```
  one  two
a  1.0    1
b  2.0    2
c  3.0    3
d  4.0    4
e  5.0    5
f  NaN    6
one    2.0
two    2.0
Name: b, dtype: float64
```

Slice Rows

It is another method to select multiple rows using **':'** operator.

```
import pandas as pd
```

```
info = {'one' : pd.Series([1, 2, 3, 4, 5], index=['a', 'b', 'c', 'd', 'e']),
        'two' : pd.Series([1, 2, 3, 4, 5, 6], index=['a', 'b', 'c', 'd', 'e', 'f'])}
df = pd.DataFrame(info)
print (df[2:5])
```

Output:

```
   one  two
c  3.0    3
d  4.0    4
e  5.0    5
```

Selection by integer location:

The rows can also be selected by passing the integer location to an **iloc** function.

Example:

```
import pandas as pd
info = {'one' : pd.Series([1, 2, 3, 4, 5], index=['a', 'b', 'c', 'd', 'e']),
        'two' : pd.Series([1, 2, 3, 4, 5, 6], index=['a', 'b', 'c', 'd', 'e', 'f'])}
df = pd.DataFrame(info)
print (df.iloc[3])
```

Output:

```
one    4.0
two    4.0
Name: d, dtype: float64
```

Pandas DataFrame.head()

The head() returns the first n rows for the object based on position. If your object has the right type of data in it, it is useful for quick testing. This method is used for returning top n (by default value 5) rows of a data frame or series.

Example:

```
import numpy as np

import pandas as pd

info = pd.DataFrame({'language':['C', 'C++', 'Python', 'Java','PHP']})

print(info.head())

print(info.head(3))
```

output:

```
  language
0        C
1      C++
2   Python
3     Java
4      PHP
  language
0        C
1      C++
2   Python
```

Example 2:

Reading the data from csv file using Pandas:

```
import pandas as pd

# making data frame

data = pd.read_csv("salary.csv")

print(data)

print(data.head())

print(data.head(2))
```

Output:

```
   age  exp  salary
0   21    1   15000
1   25    2   22000
2   34    5   27000
3   45    2   34000
4   29    5   40000
5   47   10   57000
6   56   15   90000
7   36    8   45000
8   46    9   50000
   age  exp  salary
0   21    1   15000
1   25    2   22000
2   34    5   27000
3   45    2   34000
4   29    5   40000
   age  exp  salary
0   21    1   15000
1   25    2   22000
```

Example 3:

```
import pandas as pd

# making data frame

data = pd.read_csv("salary.csv")

print(data)

print(data.tail())

print(data.tail(2))

print(data.describe())
```

Output:

```
   age  exp  salary
0   21    1   15000
1   25    2   22000
2   34    5   27000
```

```
3   45    2   34000
4   29    5   40000
5   47   10   57000
6   56   15   90000
7   36    8   45000
8   46    9   50000
  age  exp  salary
4   29    5   40000
5   47   10   57000
6   56   15   90000
7   36    8   45000
8   46    9   50000
  age  exp  salary
7   36    8   45000
8   46    9   50000
          age       exp       salary
count   9.000000   9.000000      9.000000
mean   37.666667   6.333333  42222.222222
std    11.597414   4.582576  22426.422908
min    21.000000   1.000000  15000.000000
25%    29.000000   2.000000  27000.000000
50%    36.000000   5.000000  40000.000000
75%    46.000000   9.000000  50000.000000
max    56.000000  15.000000  90000.000000
```

Pandas Dataframe.sample()

The Pandas sample() is used to select the rows and columns from the DataFrame randomly. If we want to build a model from an extensive dataset, we have to randomly choose a smaller sample of the data that is done through a function **sample**.

#Randomly selecting Rows:

```
import pandas as pd

# making data frame

data = pd.read_csv("salary.csv")

row1 = data.sample(n=2)
```

```
print(row1)
```

Output:

```
  age  exp  salary
1   25    2   22000
2   34    5   27000
```

Pandas DataFrame.shift()

If you want to shift your column or subtract the column value with the previous row value from the DataFrame, you can do it by using the **shift()** function.

Example:

```
import pandas as pd
info= pd.DataFrame({'a_data': [45, 28, 39, 32, 18],
'b_data': [26, 37, 41, 35, 45],
'c_data': [22, 19, 11, 25, 16]})
print(info)
print(info.shift(periods=2))
```

Output:

```
   a_data  b_data  c_data
0      45      26      22
1      28      37      19
2      39      41      11
3      32      35      25
4      18      45      16
   a_data  b_data  c_data
0     NaN     NaN     NaN
1     NaN     NaN     NaN
2    45.0    26.0    22.0
3    28.0    37.0    19.0
4    39.0    41.0    11.0
```

Example 2:

The example shows how to fill the missing values in the DataFrame using the **fill_value**.

```
import pandas as pd

info= pd.DataFrame({'a_data': [45, 28, 39, 32, 18],

'b_data': [26, 37, 41, 35, 45],

'c_data': [22, 19, 11, 25, 16]})

print(info)

print(info.shift(periods=2))

print(info.shift(periods=2,axis=1,fill_value=70))
```

Output:

```
   a_data  b_data  c_data
0      45      26      22
1      28      37      19
2      39      41      11
3      32      35      25
4      18      45      16
   a_data  b_data  c_data
0     NaN     NaN     NaN
1     NaN     NaN     NaN
2    45.0    26.0    22.0
3    28.0    37.0    19.0
4    39.0    41.0    11.0
   a_data  b_data  c_data
0      70      70      45
1      70      70      28
2      70      70      39
3      70      70      32
4      70      70      18
```

Pandas DataFrame.sort()

We can efficiently perform sorting in the DataFrame through different kinds:

- **By label**

- **By Actual value**

Example:

```
import pandas as pd

import numpy as np

info=pd.DataFrame(np.random.randn(10,2),index=[1,3,7,2,4,5,9,8,0,6],column
s=['col2','col1'])

print(info)
```

Output:

```
      col2      col1
1 -2.052051 -1.461066
3  0.502698 -2.079272
7 -1.296209 -2.032770
2  0.811034  1.575018
4  0.342314  0.795474
5  0.464809  0.004375
9  0.532767 -1.283186
8 -0.725044  1.173524
0 -0.716242 -0.130059
6 -1.727105 -0.142552
```

- **By label**

```
import pandas as pd

import numpy as np

info=pd.DataFrame(np.random.randn(10,2),index=[1,3,7,2,4,5,9,8,0,6],column
s=['col2','col1'])

print(info)

info2 = info.sort_index()

print(info2)
```

Output:

```
      col2      col1
1 -0.016375  0.908684
3  0.298856 -0.141335
7 -0.568217  0.121772
2  0.701895  0.556407
4  1.882900  0.672700
5  0.064719 -0.700176
9  0.271593  1.828895
8  1.273571  0.137765
0  0.670018 -0.415054
6  1.287786  0.425077
      col2      col1
0  0.670018 -0.415054
1 -0.016375  0.908684
2  0.701895  0.556407
3  0.298856 -0.141335
4  1.882900  0.672700
5  0.064719 -0.700176
6  1.287786  0.425077
7 -0.568217  0.121772
8  1.273571  0.137765
9  0.271593  1.828895
```

By Actual Value

It is another kind through which sorting can be performed in the DataFrame. Like index sorting, **sort_values()** is a method for sorting by the values.

Example:

```
import pandas as pd

import numpy as np

info=pd.DataFrame({'col1':[7,1,8,3],'col2':[8,12,4,9]})

info_2 = info.sort_values(by='col2')

print(info_2)
```

Output:

```
   col1  col2
2     8     4
0     7     8
3     3     9
1     1    12
```

Pandas DataFrame.to_excel()

We can export the DataFrame to the excel file by using the to_excel() function.

To write a single object to the excel file, we have to specify the target file name. If we want to write to multiple sheets, we need to create an **ExcelWriter** object with target filename and also need to specify the sheet in the file in which we have to write.

Example

Pandas DataFrame.transpose()

The transpose() function helps to transpose the index and columns of the dataframe. It reflects DataFrame over its main diagonal by writing the rows as columns and vice-versa.

Example:

```
# importing pandas as pd
import pandas as pd
# Creating the DataFrame
info = pd.DataFrame({"A":[8, 2, 7, None, 6],
                     "B":[4, 3, None, 9, 2],
                     "C":[17, 42, 35, 18, 24],
                     "D":[15, 18, None, 11, 12]})
# Create the index
index_ = ['Row1', 'Row2', 'Row3', 'Row4', 'Row5']
```

```
# Set the index
info.index = index_
# Print the DataFrame
print(info)
# return the transpose
result = info.transpose()
# Print the result
print(result)
```

output:

```
      A    B   C     D
Row1  8.0  4.0  17  15.0
Row2  2.0  3.0  42  18.0
Row3  7.0  NaN  35   NaN
Row4  NaN  9.0  18  11.0
Row5  6.0  2.0  24  12.0
  Row1  Row2  Row3  Row4  Row5
A   8.0   2.0   7.0   NaN   6.0
B   4.0   3.0   NaN   9.0   2.0
C  17.0  42.0  35.0  18.0  24.0
D  15.0  18.0   NaN  11.0  12.0
```